League of Legends Win Prediction

How our machine learning project works, step by step

Group 44 · Anton Andersson, Liam Jörgensen, Rasmus Rolin · Machine Learning course project

In short
Question: Can we predict which team wins a Diamond-ranked League of Legends game using only stats from the first 10 minutes?
Problem type: Classification with two classes: *blueWins* = 1 (blue team won) or 0 (blue team lost).
Models: Logistic Regression (written from scratch with NumPy) and Random Forest (scikit-learn).
Result: Both models predict the right winner in about **72%** of unseen games (Logistic Regression 71.5%, Random Forest 72.0%). They are practically equally good. Almost all of the useful information is in the gold and experience difference between the teams.

Project files and how to run

File / folder	What it is
data/high_diamond_ranked_10min.csv	The dataset, downloaded from Kaggle.
lol_win_prediction.py	All code, split into the 11 steps from the course guidelines (load, explore, preprocess, ...).
figures/	The 8 plots the code creates (used in this document).
results/results.json	Every number shown in this document, saved by the code.
requirements.txt	The Python libraries needed: numpy, pandas, matplotlib, scikit-learn.

To run: `pip install -r requirements.txt` and then `python lol_win_prediction.py`. It takes about 20 seconds. Each section below says which step in the code it explains.

1. The dataset

In the code: step 2 (load the dataset)

The dataset is *League of Legends Diamond Ranked Games (10 min)* from Kaggle. It has **9,879 games** from high-ranked players (Diamond I to Master). Each row is one game, and each value describes the game **exactly at minute 10**. There are **40 columns**: *gameId*, the target *blueWins*, and the same 19 stats for each team (columns starting with *blue…* and *red…*).

What the stats mean (for a reader who has not played the game):

Column (blue… / red…)	Meaning
WardsPlaced, WardsDestroyed	Wards are items that show part of the map. Placing them gives information; destroying the enemy's takes theirs away.
FirstBlood	1 if this team got the first kill of the game.
Kills, Deaths, Assists	Enemies killed, times the team died, and assists (helping a teammate get a kill).
Dragons, Heralds, EliteMonsters	Large neutral monsters that give the team bonuses. EliteMonsters = Dragons + Heralds.
TowersDestroyed	Enemy defence towers destroyed.
TotalGold, GoldPerMin	Gold earned. Gold buys items, which make champions stronger.
TotalExperience, AvgLevel	Experience (XP) makes champions level up and get stronger.

Column (blue... / red...)	Meaning
TotalMinionsKilled, CSPerMin	Small AI units killed in the lanes ("creep score"). The main source of gold.
TotalJungleMinionsKilled	Neutral monsters killed in the jungle (also gold and XP).
GoldDiff, ExperienceDiff	Own total minus the enemy total. For blue: positive = blue is ahead.

2. Exploring the data

In the code: step 3 (explore the data)

Before building any model we looked at the data to find problems and to understand what might predict a win.

Question	What we found	What it means for us
How big is it?	9,879 rows, 40 columns	Large enough to train and test, small enough to run in seconds.
Missing values?	0	Nothing to fill in.
Duplicate games?	0	No game is counted twice.
Data types?	All numbers (34 whole numbers, 6 decimals)	No text categories, so no encoding is needed.
Are the classes balanced?	4,930 blue wins (49.9%) and 4,949 losses	Balanced. Accuracy is a fair metric and we do not need to rebalance the data.
Irrelevant columns?	gameId	Only an ID number. Must be removed.
Redundant columns?	All 10 checks were true (see section 3)	Many columns are exact copies or formulas of other columns.
Unusual values?	Wards placed: median 16, max 276. 216 games have a team with more than 100.	Probably players spamming cheap wards. These are real games, not errors, so we keep them.

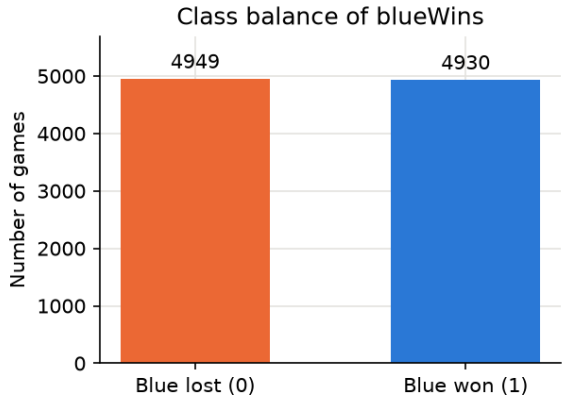


Figure 1. The two classes are almost exactly the same size.

Which stats are related to winning?

We calculated the **correlation** between every column and *blueWins*. Correlation is a number from −1 to 1: close to 1 means "when this is high, blue usually wins", close to −1 means the opposite, and 0 means no straight-line relationship. The strongest are *blueGoldDiff* (0.51) and *blueExperienceDiff* (0.49). Wards placed has almost no relationship (0.00).

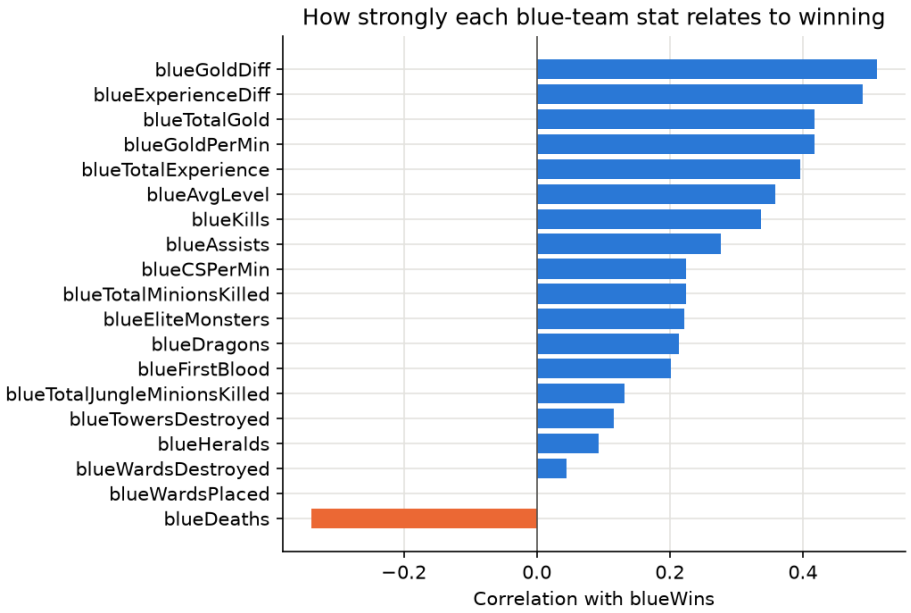


Figure 2. Correlation of each blue-team stat with blueWins. Red-team stats are the mirror image.

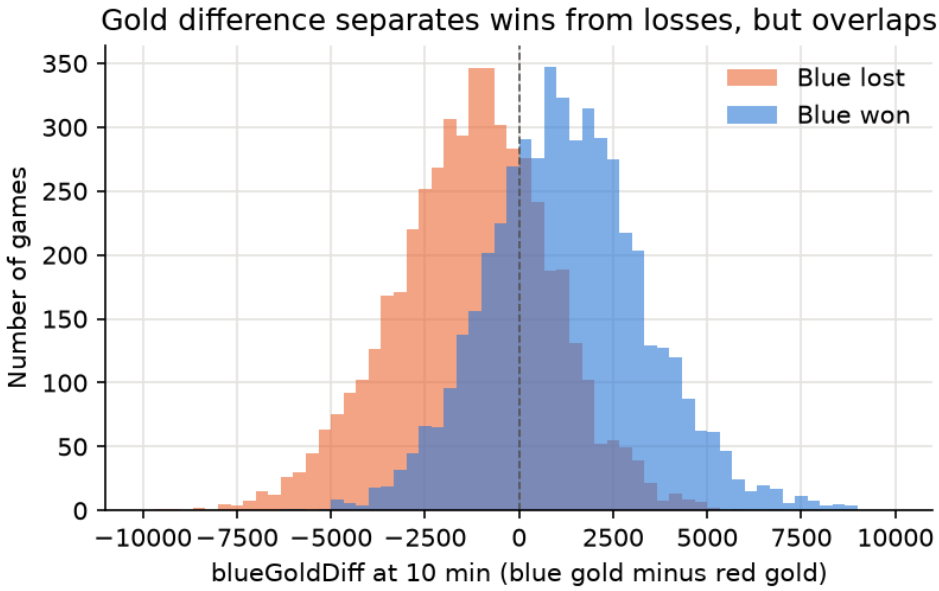


Figure 3. Gold difference at 10 minutes in games blue won (blue) and lost (orange). The groups are clearly shifted, but they overlap a lot around 0.

Figure 4 groups games by gold difference and shows how often blue won in each group. With 1,000–2,000 gold behind, blue wins 29% of games; with 1,000–2,000 ahead, 69%; with 2,000–3,000 ahead, 83%.

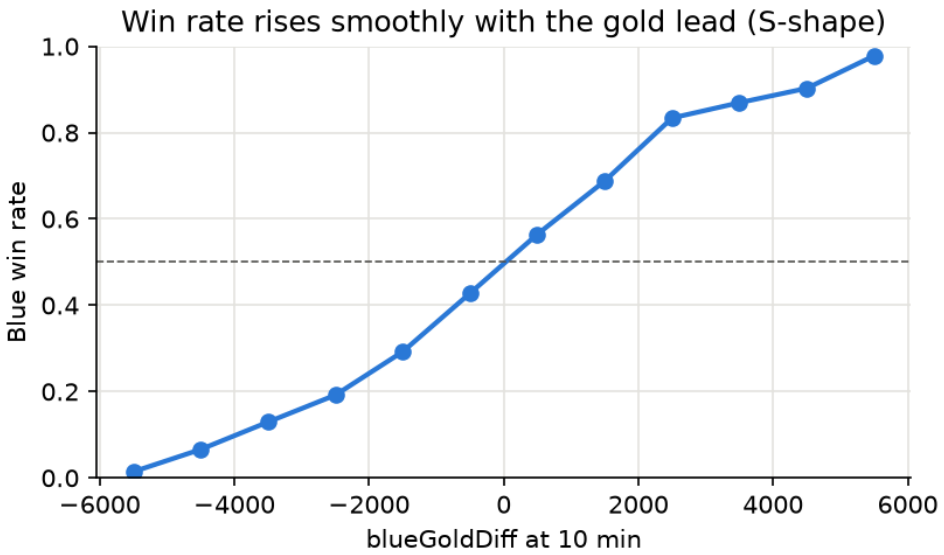


Figure 4. Blue's win rate for games grouped by gold difference.

> **Key insight**
> The curve in Figure 4 is **S-shaped**. That is exactly the shape of the sigmoid function inside Logistic Regression (section 5), which is a hint that Logistic Regression should fit this data well. It also shows the limit: 3,248 games (33%) are within ±1,000 gold, where the result is close to a coin flip.

3. Preprocessing

In the code: steps 4 and 5 (preprocess, define features and target)

What we did	Why
Removed *gameId*	It is just an ID. It cannot cause a win, but a model could still find random patterns in it.
Removed 15 redundant columns (table below)	They can be calculated exactly from columns we keep, so they add no new information. For Logistic Regression they also cause a real problem: if two columns are copies, the model can split the weight between them in endless ways, so the weights mean nothing.
Kept the differences, removed the totals	*blueGoldDiff* = *blueTotalGold* − *redTotalGold*, so keeping all three is redundant. We kept the difference because it directly measures who is ahead. Same for experience.
Did not fill in anything	There are no missing values.
Did not encode anything	All columns are already numbers. Yes/no columns (like FirstBlood) are already 0 and 1.
Kept the outliers	The extreme ward counts are real games. Scaling (section 4) stops large values from dominating.

Removed column	Reason	Removed column	Reason
redFirstBlood	= 1 − blueFirstBlood	redCSPerMin	= redTotalMinionsKilled / 10
redKills	= blueDeaths	blueEliteMonsters	= blueDragons + blueHeralds
redDeaths	= blueKills	redEliteMonsters	= redDragons + redHeralds
redGoldDiff	= −blueGoldDiff	blueTotalGold	replaced by blueGoldDiff (blue gold − red gold)
redExperienceDiff	= −blueExperienceDiff	redTotalGold	replaced by blueGoldDiff

Removed column	Reason	Removed column	Reason
blueGoldPerMin	= blueTotalGold / 10	blueTotalExperience	replaced by blueExperienceDiff (blue XP − red XP)
redGoldPerMin	= redTotalGold / 10	redTotalExperience	replaced by blueExperienceDiff
blueCSPerMin	= blueTotalMinionsKilled / 10		

Result: 23 features (inputs) and 1 target. The features are: *blueWardsPlaced*, *blueWardsDestroyed*, *blueFirstBlood*, *blueKills*, *blueDeaths*, *blueAssists*, *blueDragons*, *blueHeralds*, *blueTowersDestroyed*, *blueAvgLevel*, *blueTotalMinionsKilled*, *blueTotalJungleMinionsKilled*, *blueGoldDiff*, *blueExperienceDiff*, *redWardsPlaced*, *redWardsDestroyed*, *redAssists*, *redDragons*, *redHeralds*, *redTowersDestroyed*, *redAvgLevel*, *redTotalMinionsKilled*, *redTotalJungleMinionsKilled*.

In code: `X` = the table of 23 features, `y` = the *blueWins* column.

4. Training/test split and scaling

In the code: step 6 (split into training and test sets)

Train/test split

We split the games into **80% training (7,903 games)**, used to build the models, and **20% test (1,976 games)**, hidden until the very end. The test set plays the role of "new games the model has never seen", so it tells us how well the model would work in reality. `stratify=y` keeps the 50/50 win/loss ratio in both parts, and `random_state=42` makes the split the same every time we run the code.

Standardisation (scaling)

For each feature we compute the mean and standard deviation (std) and transform every value x like this:

$$z = (x - \text{mean}) / \text{std}$$

Afterwards every feature has mean 0 and standard deviation 1. We do this because:

- The features have very different ranges (gold difference in thousands, dragons only 0 or 1). Gradient descent (section 5) then learns slowly, because one step size does not suit all features.
- The Logistic Regression weights become comparable: a larger weight means a more important feature.

> **Important detail**
> The mean and std are calculated on the **training set only**, and the same numbers are then used on the test set. If we used the test set too, information from the "unseen" games would leak into training (**data leakage**) and our test score would look better than it really is.

5. Algorithm 1: Logistic Regression from scratch

In the code: step 7 (class LogisticRegressionScratch)

5.1 The idea in three steps

Step 1: a score. Every feature x gets a weight w, and there is one extra number b (the bias):

$$z = w_1x_1 + w_2x_2 + \ldots + w_{23}x_{23} + b$$

A positive weight means the feature pushes towards "blue wins", a negative weight towards "blue loses".

Step 2: a probability. The **sigmoid** function squeezes any score into a number between 0 and 1:

$$p = \sigma(z) = 1 / (1 + e^{-z})$$

z = 0 gives p = 0.5. A large positive z gives p close to 1, a large negative z gives p close to 0. p is the model's probability that blue wins.

Step 3: a decision. If p ≥ 0.5 we predict 1 (blue wins), otherwise 0.

Training means finding the 23 weights and the bias that make these probabilities match the real results as well as possible.

5.2 Measuring the error: the loss function

We use **binary cross-entropy** (also called log loss). For m games with real results y (0 or 1) and predicted probabilities p:

$$\text{Loss} = -(1/m) \cdot \Sigma \left[y \cdot \log(p) + (1 - y) \cdot \log(1 - p) \right]$$

In words: if blue really won (y = 1), the game adds −log(p) to the loss. That is small when p is close to 1 and very large when the model was confident and wrong (p close to 0). The same idea applies to losses with 1 − p. Training starts with all weights = 0, so every game gets p = 0.5 and the loss is log(2) ≈ 0.693.

5.3 Gradient descent

The **gradient** tells us how the loss changes if we change each weight a little. For logistic regression it has a simple form:

$$\text{gradient}_w = (1/m) \cdot X^T(p - y)$$

$$\text{gradient}_b = (1/m) \cdot \Sigma(p - y)$$

Here (p − y) is the error for each game. We then take a small step in the opposite direction, because that is where the loss goes down:

$$w \leftarrow w - \alpha \cdot \text{gradient}_w$$

$$b \leftarrow b - \alpha \cdot \text{gradient}_b$$

α is the **learning rate** (step size). One step is one **iteration**, and we repeat it 1,000 times. You can think of it as walking downhill in fog: feel which way the ground slopes, take a step down, repeat.

5.4 The code

```
def fit(self, X, y):
    n_samples, n_features = X.shape
    self.w = np.zeros(n_features)  # one weight per feature, start at 0
    self.b = 0.0                   # bias (intercept)
    self.loss_history = []

    for _ in range(self.n_iterations):
        # 1) predict a probability for every training game
        p = self.sigmoid(X @ self.w + self.b)

        # 2) measure how wrong we are (cross-entropy loss)
        eps = 1e-12  # avoids log(0)
        loss = -np.mean(y * np.log(p + eps) + (1 - y) * np.log(1 - p + eps))
        self.loss_history.append(loss)

        # 3) gradients: in which direction should w and b move?
        error = p - y
        grad_w = (X.T @ error) / n_samples
        grad_b = np.mean(error)

        # 4) take a small step in the opposite direction of the gradient
        self.w -= self.learning_rate * grad_w
        self.b -= self.learning_rate * grad_b
    return self
```

Comments 1–4 in the code match sections 5.1–5.3. `X @ self.w` computes the score z for all games at once (a matrix product). Two small safety details: `np.clip` in the sigmoid stops `np.exp` from overflowing, and `eps` stops `log(0)`. `predict_proba` returns p, and `predict` turns p into 0 or 1 with the 0.5 threshold.

5.5 Choosing the learning rate

We trained the model with four learning rates, 1,000 iterations each, and compared the final training loss:

Learning rate	0.001	0.01	0.1	1.0
Training loss after 1,000 iterations	0.5594	0.5311	0.5275	0.5262

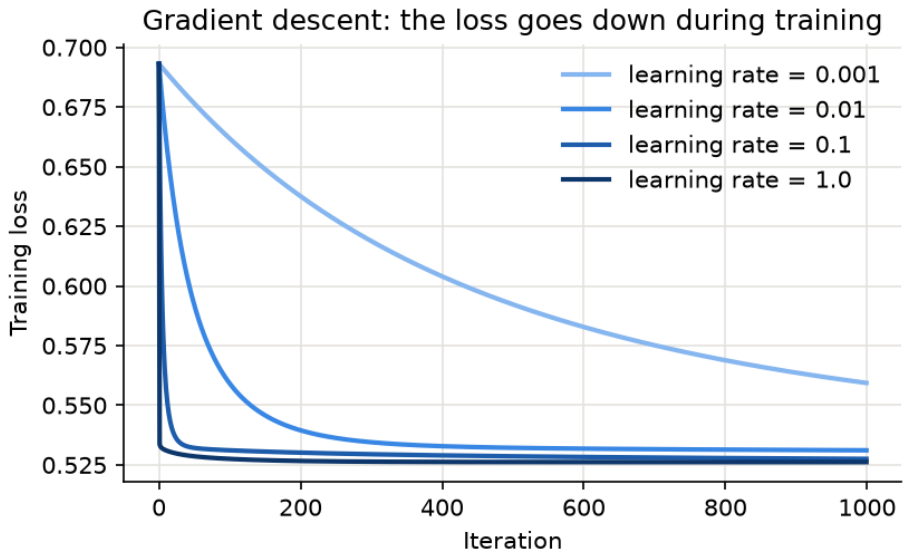


Figure 5. Training loss per iteration. A small learning rate learns very slowly.

A learning rate of 1.0 reached the lowest loss, so we used it. Such a large step only works because the features are scaled; the loss still goes down smoothly. We chose using the **training loss only**, never the test set.

Is our implementation correct?
As a one-time check (not part of the submitted code), the trained weights were compared with scikit-learn's LogisticRegression without regularisation on the same data. The largest difference in any weight was 0.006, so our gradient descent finds the same solution as the library.

6. Algorithm 2: Random Forest

In the code: step 8 (RandomForestClassifier from scikit-learn)

6.1 One decision tree

A decision tree predicts by asking yes/no questions, for example "Is blueGoldDiff > 1,500?", then "Is blueExperienceDiff > −300?", and so on until it reaches a **leaf**. The leaf's answer is the most common result among the training games that ended up there.

To choose a question, the tree tries many features and thresholds and picks the split that makes the two groups as **pure** as possible (mostly wins on one side, mostly losses on the other). Purity is measured with **Gini impurity** = $1 - (p_{win})^2 - (p_{loss})^2$, where p_{win} is the share of wins in the group. It is 0 for a pure group and 0.5 for a 50/50 group.

The problem: if a tree may grow as deep as it wants, it keeps splitting until every training game is correct. It then learns noise and random details, which is called **overfitting**.

6.2 From one tree to a forest

Our forest has **200 trees**. Two kinds of randomness make the trees different from each other:

- Each tree is trained on a **bootstrap sample**: 7,903 games drawn at random *with replacement* from the training set (some games appear twice, some not at all).
- At each split, a tree may only choose among a random subset of the features ($\sqrt{23} \approx 4$ features).

To predict, the forest combines the votes of all trees (it averages their predicted probabilities). Single trees make different mistakes, and these partly cancel out, so the forest is much more stable than one tree.

6.3 Choosing max_depth with cross-validation

max_depth is how many questions in a row a tree may ask. We tried 1 to 20 and no limit (None). To compare them **without using the test set**, we used **5-fold cross-validation** on the training set:

- Split the training set into 5 equal parts.
- Train on 4 parts and measure accuracy on the 5th.
- Repeat 5 times so that each part is used once for checking, then take the average.

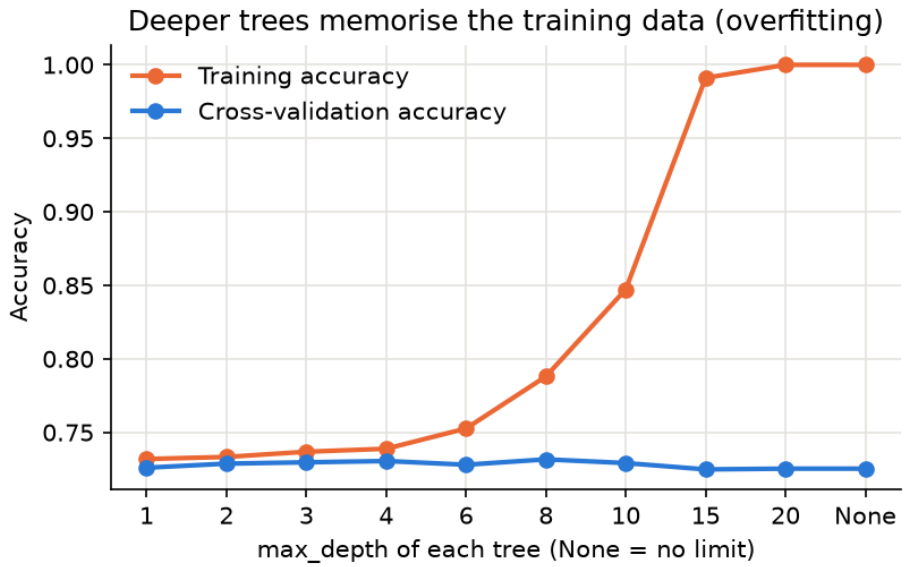


Figure 6. Training accuracy and cross-validation accuracy for each max_depth.

Training accuracy climbs to 100% for deep trees, but cross-validation accuracy stays around 73%: the extra depth only memorises the training games. The best cross-validation accuracy was at **max_depth = 8** (73.2%). All depths from 2 to 10 are within 0.4 percentage points of each other, so the choice matters very little.

Note: trees do not need scaled data, because a split like "x > 5" works the same on any scale. We gave the forest the scaled data only so that both models get the same input.

7. How we measure performance

In the code: step 9 (evaluate both models)

All final numbers are measured on the **test set**. We call "blue wins" the *positive* class. A **confusion matrix** counts the four possible outcomes:

	Predicted: blue lost	**Predicted: blue won**
Actually lost	True negative (TN), correct	False positive (FP), wrong
Actually won	False negative (FN), wrong	True positive (TP), correct

Metric	**Formula**	**Question it answers**
Accuracy	(TP + TN) / all games	How many games did the model predict correctly?
Precision	TP / (TP + FP)	When the model says "blue wins", how often is that true?
Recall	TP / (TP + FN)	Of all games blue really won, how many did the model find?
F1-score	2 · Precision · Recall / (Precision + Recall)	One number that balances precision and recall.

Because the classes are balanced, accuracy is a fair main metric. We also compare **training accuracy with test accuracy** to detect overfitting, and compare with two simple **baselines**: always guessing the most common class, and the rule "the team with more gold at 10 minutes wins".

8. Results

In the code: steps 10 and 11 (compare the results, visualisations)

Metric (test set unless stated)	Logistic Regression	Random Forest (max_depth 8)	Random Forest (no depth limit)
Training accuracy	73.8%	78.8%	100.0%
Test accuracy	**71.5%**	**72.0%**	72.1%
Precision	71.1%	72.4%	72.6%
Recall	72.3%	70.9%	70.8%
F1-score	71.7%	71.6%	71.7%

Baseline	Test accuracy
Always guess the most common class ("blue loses")	50.1%
"The team with more gold at 10 minutes wins"	71.8%

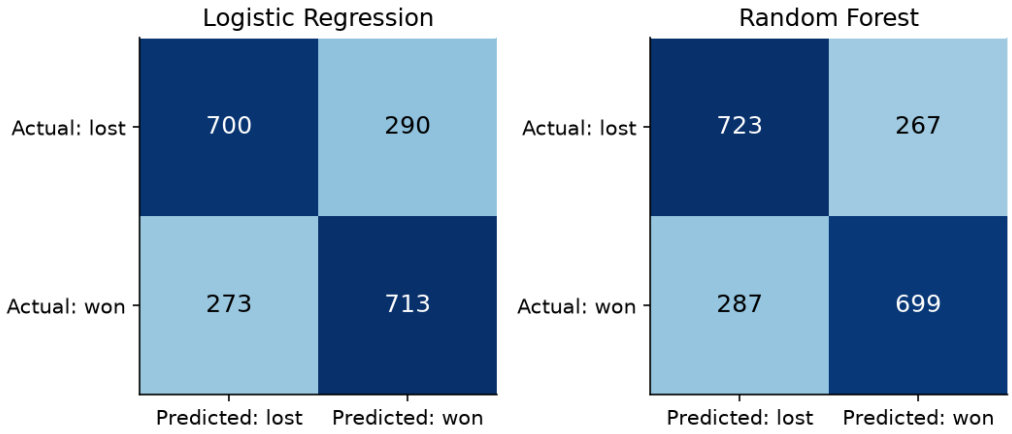


Figure 7. Confusion matrices on the 1,976 test games.

Logistic Regression predicted **1,413** of 1,976 test games correctly, and Random Forest **1,422**, a difference of 9 games. Both make about as many mistakes in each direction.

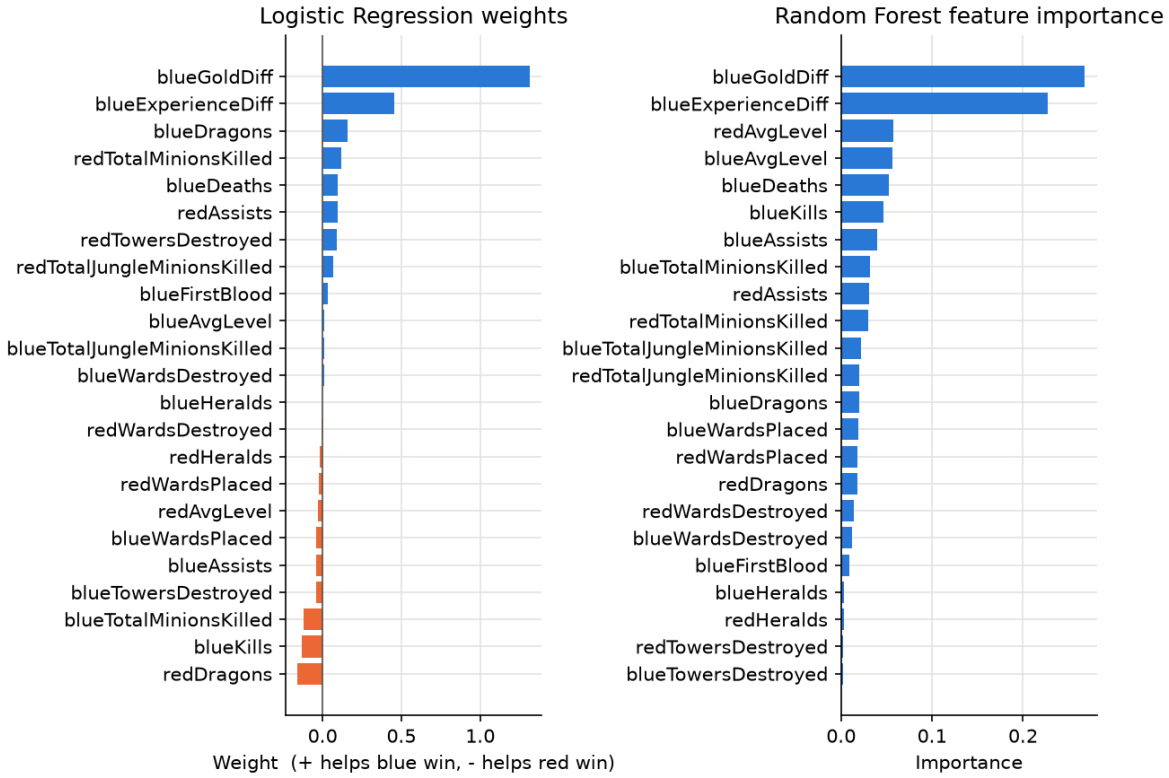


Figure 8. Left: Logistic Regression weights (on scaled features). Right: Random Forest feature importance (how much each feature improved the splits).

Both models agree that **gold difference** and **experience difference** matter by far the most. Wards, heralds and towers matter very little at 10 minutes.

9. Comparison and discussion

Which model is better?

Random Forest had a slightly higher test accuracy (72.0% vs 71.5%) and precision. Logistic Regression had a slightly higher recall and F1-score. The difference is only 9 games out of 1,976, and a different random split could easily reverse it, so we consider the two models **equally good**.

Why are they so similar?

The relationship between the features and winning is close to a straight line in the score z: more gold and XP give a steadily higher chance to win (the S-curve in Figure 4). Logistic Regression is built for exactly this. Random Forest's strength is finding complicated patterns and interactions between features, and there are few such patterns here to find.

Why only about 72%?

A game lasts around 30 minutes, so after 10 minutes a lot can still happen: team fights, mistakes and comebacks. The data also says nothing about which champions were picked or how good each player is. In 33% of the games the gold difference is under 1,000, and those games are close to a coin flip. So a much higher accuracy is probably not possible with these columns.

The gold baseline

The simple rule "more gold wins" gets 71.8%, almost as good as both models. This shows that most of the information is in the gold difference. The models add only a little on top, mainly from the experience difference and dragons.

Overfitting

- **Logistic Regression:** training 73.8% vs test 71.5%. A small gap: a model with only 24 numbers cannot memorise thousands of games.
- **Random Forest, no depth limit:** training 100.0% vs test 72.1%. Clear overfitting: it has memorised the training games. The test score is still fine because averaging many different trees cancels much of the noise.
- **Random Forest, max_depth 8:** training 78.8% vs test 72.0%. Some overfitting, but limited.

Reading the Logistic Regression weights

blueGoldDiff has the largest weight (+1.32), then *blueExperienceDiff* (+0.46). *blueDragons* (+0.16) and *redDragons* (-0.16) also matter, because dragons give lasting bonuses that do not show up in gold.

A surprise: *blueKills* has a **negative** weight (-0.13), even though kills are positively correlated with winning (0.34). The reason is that the gold from kills is already included in *blueGoldDiff*. If the gold difference is held fixed, more kills means less of the gold came from other sources such as minions. So when features overlap, one weight should not be read on its own.

Limitations

- We used one test split. With another random split, accuracy could move by about 1 percentage point (cross-validation on the training set gave about 73%, slightly higher than the test score).
- The data only covers Diamond-level games and a single moment (minute 10). The results may differ for other ranks.
- There is no information about champions, players or what happens after minute 10.

10. Conclusion and future work

We predicted the winner of Diamond-ranked League of Legends games using stats from the first 10 minutes. After removing the ID column and 15 redundant columns, we trained a Logistic Regression that we implemented ourselves with gradient descent, and a Random Forest tuned with 5-fold cross-validation.

Both models reach about **72% accuracy** on unseen games: far better than guessing (50.1%), but only slightly better than the rule "more gold wins" (71.8%). Random Forest was 0.46 percentage points better on test accuracy, which is too small to be a real difference. Logistic Regression is simpler, faster and easier to explain, so we would choose it for this problem.

Future work:

- Add data from later in the game (for example minute 15) or champion picks.
- Try other models such as gradient boosting.
- Repeat the train/test split several times and average the results for a fairer comparison.
- Try regularisation (an extra term in the loss that keeps the weights small).

11. Questions you may get at the presentation

Question	Short answer
Why is this a classification problem?	The target blueWins has only two values, 0 or 1.
Why does Logistic Regression suit this data?	It outputs a probability between 0 and 1, and the win rate rises in an S-shape with gold difference (Figure 4), which is the shape of the sigmoid.
Why did you remove so many columns?	They are exact copies or formulas of columns we kept, so they add no information and make the Logistic Regression weights impossible to interpret.
Why scale after the split, not before?	So no information from the test set leaks into training (data leakage).
What does the learning rate do?	It is the step size in gradient descent. Too small learns very slowly; too large can jump past the minimum.
Why do all weights start at 0?	The cross-entropy loss for logistic regression is bowl-shaped (convex), so there is only one minimum and the starting point does not change the result. With all weights at 0, every game starts at p = 0.5.
Why is the deep forest 100% correct on training data?	Deep trees keep splitting until every training game is classified correctly. That is memorising (overfitting), which is why we chose the depth with cross-validation.
Why does blueKills have a negative weight?	Kill gold is already counted in blueGoldDiff. With the gold difference held fixed, more kills means less gold from other sources.
Which model would you choose?	They perform almost equally well, so Logistic Regression: it is simpler, faster and easier to explain.